

UNIT 4: MEMORY MANAGEMENT • ULTRA RECALL NOTES

1. MEMORY HIERARCHY & ADDRESS SPACES

Memory Hierarchy & Latency

- **Speed Order:** Registers (<1ns) > L1/L2/L3 Cache (1-10ns) > RAM (50-100ns) > SSD/NVMe (μs) > HDD (ms).
- **Resident Monitor:** Early OS batch prototype; uses *Fence Register* to isolate OS code from user programs.
- **Logical (Virtual) Address:** CPU generates at execution.
- **Physical (Real) Address:** RAM hardware pin address.

Address Binding: 3 Crucial Phases

- **Compile Time:** Generates *Absolute Code*. Starting address fix hota hai; move karne par scratch recompile!
- **Load Time:** Generates *Relocatable Code*. Offset relative to 0; loader final RAM base address bind karta hai.
- **Execution Time:** Dynamic relocation via hardware MMU. Process runtime par RAM me swap/relocate ho sakta hai.

$$\text{Physical Addr} = \text{Logical Addr} + \text{Base Register}$$

2. DYNAMIC LOADING & LINKING

Dynamic Loading

- Routines memory me tabhi aate hain jab unhe actually call kiya jaye (on-demand). Unused error routines memory waste nahi karte.

Dynamic Linking (Shared Libs .so / .dll)

- Binary me actual code ke bajaye chhota **Stub** embed hota hai. Shared library RAM me ek baar load hoti hai aur sabhi share karte hain.

3. CONTIGUOUS ALLOCATION, FRAGMENTATION & STRATEGIES

Fixed vs Variable Partitions

Feature	Fixed (MFT)	Variable (MVT)
Partitioning	Boot time fixed slots	Dynamic on process request
Multiprogramming	Strictly partition count	Dynamic (RAM capacity)
Fragmentation	Internal Fragmentation	External Fragmentation

- **Internal Frag:** Partition space unused by smaller process.
- **External Frag:** Total free memory sufficient hai par non-contiguous hone se allocation fail ho jati hai (*50% Rule*).
- **Compaction:** Processes ko relocate karke ek bada hole banana; **requires Execution-Time Relocation!**

Dynamic Storage Allocation Strategies

- **First Fit:** Pehla hole jo process size satisfy kare. *Sabse Tez (Fastest)*, shuru me chhote fragments chhodta hai.
- **Best Fit:** Smallest hole jo satisfy kare. Badi holes preserve rehti hain par *smallest useless slivers* create hote hain (Worst external frag). $O(n)$.
- **Worst Fit:** Sabse bada available hole allocate karo. Leftover hole bada rehta hai, par largest blocks jaldi exhaust hote hain. $O(n)$.
- **Next Fit:** Circular First Fit from last allocation pointer.

$$\text{Hardware Bounds: If } L < \text{Limit} \Rightarrow \text{Base} + L; \text{ Else Trap!}$$

4. NON-CONTIGUOUS PAGING & ADDRESS TRANSLATION

Paging Principles & Address Math

- RAM is split into **Frames**; Process is split into **Pages**.
- **Invariant: Frame Size = Page Size = 2^n bytes.**
- **External Fragmentation = ZERO (Completely eliminated!)**
- Internal Fragmentation: Last page only (Avg = $\text{Page Size} / 2$).
- Logical Address = $[p : m - n \text{ bits}] \mid [d : n \text{ bits}]$.
- Page Table index p yields Frame $f \Rightarrow \text{Physical} = [f \parallel d]$.

$$\text{Physical Addr} = (f \times \text{FrameSize}) + d$$

Pure Paging Latency Bottleneck

- Pure paging me har access ke liye RAM ko **DO BAAR** access karna padta hai:
- 1. Page Table entry read karne ke liye (RAM access 1).
- 2. Actual data / instruction read karne ke liye (RAM access 2).
- Latency penalty = **100% Slowdown ($2 \times t_{\text{RAM}}$)**.

$$\text{Pure Paging Access Time} = 2 \times t_{\text{RAM}}$$

5. TLB (TRANSLATION LOOKASIDE BUFFER) & EMAT

Hardware Associative Cache & Effective Memory Access Time (EMAT)

- **TLB:** High-speed associative cache on CPU chip (1-10ns). Fast parallel search for $\langle p, f \rangle$.
- **TLB Hit (h):** Time = $t_{\text{TLB}} + t_{\text{RAM}}$. Direct physical frame read.
- **TLB Miss (1 - h):** Time = $t_{\text{TLB}} + 2 \times t_{\text{RAM}}$. Read table from RAM, update TLB, then read data.

$$\text{EMAT} = h(t_{\text{TLB}} + t_{\text{RAM}}) + (1-h)(t_{\text{TLB}} + 2t_{\text{RAM}})$$

$$\text{EMAT} = t_{\text{TLB}} + (2 - h) \times t_{\text{RAM}}$$

- **Solved Quick:** If $t_{\text{RAM}}=100\text{ns}$, $t_{\text{TLB}}=20\text{ns}$, $h=0.8 \Rightarrow \text{EMAT} = 0.8(120) + 0.2(220) = \mathbf{140 \text{ ns}}$. (At $h=0.98 \Rightarrow \mathbf{122 \text{ ns}}$).

6. MULTILEVEL & INVERTED PAGE TABLES

Two-Level Hierarchical Paging

- 32-bit CPU with 4KB pages needs $1\text{M entries} \times 4\text{B} = 4\text{MB}$ **contiguous table per process!**
- Two-Level Split:** $[p1 : 10 \text{ bits}] \mid [p2 : 10 \text{ bits}] \mid [d : 12 \text{ bits}]$.
- Outer page table fits in 1 page (4KB). Unused inner tables RAM me load nahi hote (huge RAM savings!).
- Miss Penalty:** k-level paging me miss par **(k + 1) RAM accesses** lagte hain (Two-level = 3 RAM accesses).

Inverted Page Table

- Poore system me **strictly EK TABLE** hoti hai, indexed by physical Frame Number i.
- Entry = $\langle \text{Process ID (PID)}, \text{Page Number (p)} \rangle$.
- Pro:** Memory overhead physical RAM ke size par depend karta hai, virtual space par nahi (Extremely small).
- Con:** Slow search (requires Hash Table); shared memory difficult.

7. SEGMENTATION & PAGED SEGMENTATION

Segmentation Architecture <s, d>

- User/Programmer view: Variable size modules (Code, Stack, Heap).
- Segment Table:** Stores *Base* (physical start) and *Limit* (length).
- Hardware Bounds Check:**
 - If $(d \geq \text{Limit}) \Rightarrow$ **TRAP: Segmentation Fault!**
 - If $(d < \text{Limit}) \Rightarrow$ **Physical Addr = Base + d.**

Paging vs Segmentation vs Hybrid

Property	Paging	Segmentation
Block Size	Fixed (4KB)	Variable
Defined By	Hardware/OS	Programmer/Compiler
Internal Frag	Yes (last page)	Zero
External Frag	Zero	Yes

- Paged Segmentation:** Hybrid; segments are internally paged (Combines logical protection with zero external frag).

8. VIRTUAL MEMORY & DEMAND PAGING

Demand Paging & Valid-Invalid Bit

- Virtual memory allows process size > physical RAM.
- Demand Paging:** Pages loaded into RAM only when demanded by CPU (*Lazy Swapper / Pager*).
- Page Table (V/I) Bit:**
 - 'v' (Valid = 1): Page is physically in RAM.
 - 'i' (Invalid = 0): Page is not in RAM (On disk swap space).
- Accessing invalid page triggers **PAGE FAULT TRAP!**

6-Step Page Fault Handling Sequence

- MMU detects 'i' bit $\Rightarrow$ Hardware Trap to OS.
- OS kernel saves user registers & state into PCB.
- OS checks legality; if legal, finds free frame in RAM.
- Issue disk I/O to swap-in page from backing store.
- Write frame # in Page Table; flip bit to 'v' (Valid).
- OS restarts trapped instruction from exact PC point.

9. EFFECTIVE ACCESS TIME (EAT) UNDER DEMAND PAGING

$$\text{EAT} = (1 - p) \times t_{\text{RAM}} + p \times (\text{Fault Service Time})$$

- $t_{\text{RAM}} = 100 - 200 \text{ ns}$ | Fault Service Time = $8 - 10 \text{ ms}$ (8,000,000 ns).
- Disk access is **100,000× slower than RAM!**

Numerical Rule of Thumb:

- For < 10% performance degradation: $\frac{\Delta \text{Allowed Time}}{\text{Fault Time}} \approx \frac{20 \times 10^6 \text{ ns}}{8 \times 10^6 \text{ ns}} = \frac{1}{400,000} = 0.00025\%$
- System must have fewer than 1 fault per 400,000 references!

10. FIFO PAGE REPLACEMENT & BELADY'S ANOMALY

FIFO Algorithm & Dirty Bit

- FIFO:** Replaces oldest page in RAM. Implemented via circular queue.
- Dirty (Modify) Bit:** If 0 (Clean) $\Rightarrow$ discard without disk write; If 1 (Dirty) $\Rightarrow$ must write back to swap space before evicting.

Belady's Anomaly Proof (AKTU Classic)

- Phenomenon:** Frames badhane par page faults badh jate hain!
- String:** 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- 3 Frames $\Rightarrow$ **9 Page Faults.**
- 4 Frames $\Rightarrow$ **10 Page Faults! (Faults Badh Gaye)**
- Why:** FIFO is NOT a Stack Algorithm ($S_n \not\subseteq S_{n+1}$).

11. OPTIMAL & LRU PAGE REPLACEMENT (STACK ALGORITHMS)

Optimal (OPT / MIN) Algorithm

- **Rule:** Replace page that will not be used for longest time in future.
- Guarantees **lowest possible page fault rate**.
- Theoretical benchmark; impossible in real OS (needs future oracle).
- **No Belady's Anomaly (Stack Algorithm).**

Least Recently Used (LRU) Algorithm

- **Rule:** Replace page that has not been used for longest time in past.
- Approximates OPT via temporal locality. **No Belady's Anomaly!**
- **Implementation:**
 - 1. Logical Clock Counter: Min counter value victim ($O(n)$ search).
 - 2. Doubly Linked Stack: Most recent to Top; Bottom = victim ($O(1)$).

12. CLOCK (SECOND CHANCE) & COUNTING ALGORITHMS

Second Chance (Clock) Replacement

- Circular queue + Clock Pointer + **Reference Bit (R)**.
- If $R == 0 \Rightarrow$ Replace page (Victim found!).
- If $R == 1 \Rightarrow$ Reset $R = 0$, advance pointer (Second chance!).
- Degenerates to FIFO if all bits are 1.

Enhanced Clock & Counting (LFU/MFU)

- **Enhanced Pair $\langle R, M \rangle$:** (0,0) Best victim > (0,1) Dirty > (1,0) Clean recent > (1,1) Worst candidate.
- **LFU:** Smallest reference count replaced.
- **MFU:** Highest count replaced (protects new arrivals).

13. THRASHING & WORKING-SET MODEL

Thrashing Cause & Working-Set Model

- **Thrashing:** CPU utilization crashes near 0% because system spends 99% time swapping pages between disk and RAM.
- **Root Cause:** High Degree of Multiprogramming $\Rightarrow$ Frames per process fall below locality $\Rightarrow$ continuous faults.
- **Working-Set Model:** Parameter Δ (Window).
 - $WSS_i =$ pages accessed in last Δ refs.
 - Demand $D = \sum WSS_i$.
 - **If $D > \text{Total Frames } M \Rightarrow$ Thrashing occurs!**

Page-Fault Frequency (PFF) Strategy

- Dynamic self-regulating frame allocation.
- If Fault Rate > Upper Threshold $\Rightarrow$ Allocate extra frames to process.
- If Fault Rate < Lower Threshold $\Rightarrow$ Remove surplus frames from process.
- If Upper exceeded and no frames free $\Rightarrow$ Swap-out process.

14. LOCALITY OF REFERENCE & CACHE ORGANIZATION

Locality of Reference

- **Temporal Locality:** Same location accessed again soon (Loops, loop index counters, subroutines).
- **Spatial Locality:** Adjacent memory locations accessed soon (Array sequential access, code stream).

Cache Mappings & AMAT

- **Direct:** Line = Block mod N. Fast, high conflict misses.
- **Fully Associative:** Any block in any line. Zero conflict.
- **k-Way Set:** Set = Block mod NumSets. Best balance.

$$AMAT = \text{Hit Time} + (\text{Miss Rate} \times \text{Miss Penalty})$$

15. MASTER FORMULA TABLE & TECHNICAL INTERVIEW QUICK RECALL

Concept / Metric	Formula / Condition	Key Insight
Paging Address	Physical = $(f \times \text{FrameSize}) + d$	External Frag = 0; Internal = last page
TLB EMAT	$EMAT = h(t_{TLB} + t_{RAM}) + (1-h)(t_{TLB} + 2 \times t_{RAM})$	Two-level Miss = $t_{TLB} + 3 \times t_{RAM}$
Demand Paging EAT	$EAT = (1-p) \times t_{RAM} + p \times (t_{Fault})$	p must be < 0.00025% for 10% slowdown
Segmentation	If $d < \text{Limit} \Rightarrow \text{Base} + d$; Else TRAP!	Programmer view; Internal Frag = 0
Thrashing	Thrashing if Demand $D = \sum WSS_i > M$	Prevent via PFF or swapping out process
Belady's Anomaly	FIFO fails subset property $S_n \subseteq S_{n+1}$	LRU and OPT never suffer anomaly
COW & ASID	Copy-On-Write on fork(); ASID tags in TLB	Avoids TLB flush on context switch